

วิธีใช้ Prompt ให้ได้ผลจริงที่สุด

ไม่ควรส่ง Prompt 40 หน้าทั้งหมดแล้วสั่งว่า “สร้างให้เสร็จ” เพียงครั้งเดียว แนะนำลำดับดังนี้:

คำสั่งที่ 1

อ่านข้อกำหนดทั้งหมด แต่ยังไม่ต้องเขียนโค้ด ให้สรุปสถาปัตยกรรม Game State, Screen Flow, Input Abstraction, Asset Manifest และ LocalStorage Schema ก่อน

คำสั่งที่ 2

สร้างเวอร์ชัน PO ที่เล่นด้วย Mouse และ Touch ได้ครบ 5 ด้าน โดยยังไม่เพิ่ม MediaPlayer, เสียงบรรยาย และเอฟเฟกต์ P2

คำสั่งที่ 3

ตรวจสอบไฟล์ PO ตาม Acceptance Criteria แก่ SyntaxError, ReferenceError, ID ที่ไม่มีจริง, Event Listener ซ้ำ, Timer ซ้ำ และ Game Loop ซ้อน

คำสั่งที่ 4

เพิ่ม MediaPlayer Hands และโหมด AR โดยห้ามเปลี่ยน Gameplay Logic ของ Mouse และ Touch

คำสั่งที่ 5

เพิ่มระบบ Skill, Audio, Speech Synthesis, Leaderboard และเปลี่ยนโหมดระหว่างเล่น

คำสั่งที่ 6

ตรวจสอบโค้ดทั้งไฟล์ตาม Acceptance Criteria ทุกข้อ และรายงานข้อที่ยังยืนยันไม่ได้ก่อนแก้ไข

คำสั่งที่ 7

เพิ่มเสียงบรรยายตัวละครและพื้นหลังแต่ละด่าน
ให้นำไฟล์ [index.html](#) เวอร์ชันล่าสุดที่ผ่านการตรวจสอบตาม Acceptance Criteria แล้วมาปรับปรุง โดยห้ามสร้างเกมใหม่ทั้งหมด และห้ามลบหรือเปลี่ยน Gameplay Logic เดิม
ให้เพิ่มระบบเสียงบรรยายภาษาไทยและภาพพื้นหลังตามข้อกำหนดต่อไป

1. ข้อกำหนดระบบเสียงบรรยาย

ให้ใช้ [SpeechSynthesis API](#) และฟังก์ชันเสียงบรรยายส่วนกลางที่มีอยู่ในเกม เช่น:
`speakThai(text, options)`

หากยังไม่มี ให้สร้างฟังก์ชันกลางสำหรับเสียงบรรยาย โดยต้องมีคุณสมบัติดังนี้

- กำหนดภาษาเป็น `th-TH`
- ตรวจสอบว่า Browser รองรับ [speechSynthesis](#)
- ตรวจสอบว่า `gameState.narrationEnabled` เปิดอยู่
- ก่อนเล่นข้อความใหม่ ให้ยกเลิกข้อความเดิมด้วย [speechSynthesis.cancel\(\)](#)
- ป้องกันเสียงซ้อนกัน
- ป้องกันข้อความเดียวกันเล่นซ้ำอย่างรวดเร็ว
- ใช้ระบบ Debounce หรือ Cooldown ประมาณ 800–1,200 มิลลิวินาที
- เมื่อเปลี่ยนหน้าจอ ให้กดหรือยกเลิกเสียงเดิมที่ยังเล่นอยู่
- หาก Browser ไม่มีเสียงภาษาไทย ให้เลือกเสียงค่าเริ่มต้นของระบบ
- หาก Speech Synthesis ใช้งานไม่ได้ เกมต้องทำงานต่อได้ตามปกติ
- ห้ามแสดง `alert()`
- หากเกิดข้อผิดพลาด ให้ใช้ Toast ภายในเกม

- การปิดเสียงบรรยายต้องไม่ปิดเสียงเอฟเฟกต์ของเกม
- การเปิด-ปิดเสียงบรรยายต้องบันทึกใน LocalStorage

ให้ใช้ฟังก์ชันกลางเพียงชุดเดียว ห้ามสร้างระบบเสียงบรรยายแยกในแต่ละหน้าจอ

2. เสียงต้อนรับในหน้า Start Screen

เมื่อเข้าสู่หน้า Start Screen ให้เล่นเสียงบรรยายภาษาไทยว่า:

“ยินดีต้อนรับ เข้าสู่เกม อันเตอร์ส สองจุดห้า หน่วยล่าท้าฝุ่น”

เสียงต้องเล่นเพียงหนึ่งครั้งต่อการเข้าสู่หน้า Start Screen แต่ละครั้ง และต้องไม่เล่นซ้ำจาก Game Loop หรือการ Resize หน้าจอ

เนื่องจาก Browser บางชนิดอาจป้องกันเสียงที่เล่นอัตโนมัติ ให้ดำเนินการดังนี้

1. พยายามเล่นเสียงเมื่อหน้า Start Screen แสดงสมบูรณ์
2. หาก Browser ไม่อนุญาต ให้เก็บสถานะว่ามีเสียงต้อนรับรอเล่น
3. เล่นเสียงทันทีเมื่อผู้ใช้คลิก ตะตะ หรือกดปุ่มครั้งแรกบนหน้า Start Screen
4. หลังเล่นสำเร็จแล้ว ให้ลบ Event Listener ที่ใช้รอการโต้ตอบ
5. ห้ามทำให้ปุ่มเริ่มเกมหรือปุ่มอื่นกดไม่ได้

ให้เพิ่มตัวแปรป้องกันการเล่นซ้ำ เช่น:

```
let welcomeNarrationPlayed = false;  
let pendingWelcomeNarration = false;
```

เมื่อออกจากหน้า Start Screen ให้ยกเลิกเสียงต้อนรับที่ยังเล่นอยู่

3. พื้นหลังหน้า Start Screen

ให้ใช้ภาพพื้นหลัง:

Bg/Poster.png

ข้อกำหนดการแสดงผลภาพ:

- ใช้ Relative Path เท่านั้น
- ห้ามขึ้นต้นด้วย /
- ต้องตรวจสอบตัวพิมพ์เล็กและใหญ่ให้ตรงกับชื่อไฟล์จริง
- แสดงภาพเต็มพื้นที่หน้าจอ
- จัดตำแหน่งภาพกึ่งกลาง
- ใช้ลักษณะเทียบเท่า `background-size: cover`
- รองรับ Desktop, Notebook, Tablet, iPad และ Mobile
- รองรับ Portrait และ Landscape
- ต้องไม่ยัดหรือบิดสัดส่วนของภาพ
- เพิ่ม Overlay สีเข้มแบบโปร่งแสงเพื่อให้ชื่อเกมและปุ่มอ่านง่าย
- ปุ่มและข้อความต้องมี Contrast เพียงพอ
- หากรูปภาพโหลดไม่สำเร็จ ให้ใช้พื้นหลัง Gradient เดิมของเกมเป็น Fallback
- ห้ามแสดง Broken Image Icon

ตัวอย่างแนวทาง CSS:

```
.start-screen {  
  background-image:  
    linear-gradient(rgba(3, 8, 24, 0.42), rgba(3, 8, 24, 0.72)),
```

```
url("Bg/Poster.png");
background-size: cover;
background-position: center;
background-repeat: no-repeat;
}
```

ให้ปรับ Selector ให้ตรงกับโครงสร้าง HTML จริงของเกม

4. เสียงบรรยายหน้าเลือกตัวละคร

ในหน้า Character Select เมื่อผู้ใช้ชี้ เลือก โฟกัส หรือแตะการ์ดตัวละคร ให้เล่นเสียงบรรยายตามตัวละครดังนี้

Sky – สกาย

ให้พูดว่า:

“สกาย ฉายา สายโล่ เอ็นเก้าห้า”

Zee – ซี

ให้พูดว่า:

“ซี ฉายา จอมลุย”

Bubble – บับเบิล

ให้พูดว่า:

“บับเบิล ฉายา บอทิฟิทัสปอด”

ให้รองรับการควบคุมดังนี้

เมาส์

- ใช้เหตุการณ์ **pointerenter** หรือ **mouseenter**
- เล่นเสียงเมื่อ Pointer เข้าไปอยู่บนการ์ดตัวละคร
- ห้ามเล่นเสียงซ้ำต่อเนื่องขณะ Pointer ยังอยู่บนการ์ดเดิม
- เมื่อ Pointer ย้ายไปตัวละครใหม่ ให้หยุดข้อความเดิมและพูดข้อความของตัวละครใหม่

หน้าจอสัมผัส

- ใช้ **pointerdown** หรือ **touchstart**
- เมื่อแตะการ์ด ให้เลือกตัวละครและเล่นเสียงของตัวละครนั้น
- ห้ามบังคับให้ผู้เล่นแตะสองครั้งเพื่อเลือก
- การลากนิ้วผ่านการ์ดต้องไม่ทำให้เสียงหลายตัวละครเล่นซ้อนกัน

Keyboard และ Accessibility

- ใช้ **focusin** หรือ **focus**
- เมื่อใช้ Tab โฟกัสไปยังการ์ด ให้เล่นเสียงของตัวละครนั้น
- การ์ดตัวละครต้องสามารถรับ Focus ได้
- ต้องมี **aria-label** ที่มีชื่อตัวละคร ฉายา HP และทักษะ
- ปุ่มหรือการ์ดต้องเลือกได้ด้วย Enter และ Space

ให้เพิ่มระบบ Debounce และจำตัวละครที่พูดล่าสุด เช่น:

```
let lastNarratedHero = null;
```

```
let lastHeroNarrationTime = 0;
```

ห้ามผูก Event Listener ซ้ำทุกครั้งที่เปิดหน้า Character Select หากมี Listener อยู่แล้ว
เมื่อออกจากหน้า Character Select ให้ยกเลิกเสียงที่ยังเล่นอยู่

5. พื้นหลังด่านที่ 1

หน้าด่านที่ 1:

ฝุ่นเริ่มคุกคาม

Dust Alert — Yellow Zone

ให้ใช้ภาพพื้นหลัง:

Bg/Yellow-Zone.png

ข้อกำหนด:

- ภาพเต็มพื้นที่ Gameplay
 - ใช้ **cover**
 - จัดกึ่งกลาง
 - เพิ่ม Overlay เพื่อให้ไอเท็ม คำแนะนำ Cursor และ HUD มองเห็นชัด
 - Overlay ต้องไม่เข้มจนภาพพื้นหลังหาย
 - หากโหลดภาพไม่ได้ ให้ใช้พื้นหลังจำลอง Yellow Zone เดิม
 - ห้ามให้พื้นหลังรับ Pointer Event
 - พื้นหลังต้องอยู่หลัง Canvas และวัตถุ Gameplay
 - ต้องไม่กระทบ Collision Detection หรือฟิสิกส์ Input
-

6. พื้นหลังด่านที่ 2

หน้าด่านที่ 2:

ปีศาจฝุ่นจิ๋ว

Tiny Dust Monsters

ให้ใช้ภาพพื้นหลัง:

Bg/Monsters.png

ข้อกำหนด:

- แสดงเต็มพื้นที่ Gameplay
 - ใช้ **background-size: cover**
 - จัดกึ่งกลาง
 - เพิ่ม Overlay สีม่วง ดำ หรือแดงแบบโปร่งแสงตามความเหมาะสม
 - ศัตรูต้องมองเห็นเด่นกว่าพื้นหลัง
 - ห้ามใช้ Overlay ที่ทำให้ผู้เล่นแยกศัตรูแต่ละชนิดไม่ได้
 - หากภาพโหลดไม่ได้ ให้ใช้ Gradient และ Fog เดิมเป็น Fallback
 - ต้องไม่กระทบระบบ Spawn, Movement, Hitbox และ Collision
-

7. พื้นหลังด่านที่ 3

หน้าด่านที่ 3:

ราชาฝุ่นจิ๋ว

PM 2.5 Mega Boss

ให้ใช้ภาพพื้นหลัง:

ข้อกำหนด:

- แสดงเต็มพื้นที่ Gameplay
 - ใช้ **cover**
 - จัดกึ่งกลาง
 - เพิ่ม Overlay สีดำหรือม่วงเข้มแบบโปร่งแสง
 - บอสและ Projectile ต้องมองเห็นชัดเจน
 - สัญญาณเตือนการโจมตีต้องไม่กลืนไปกับพื้นหลัง
 - หากภาพโหลดไม่ได้ ให้ใช้ Boss Arena Gradient เดิม
 - ห้ามทำให้ Boss HP, Player HP, Timer หรือ Cursor อ่านยาก
 - ต้องไม่กระทบ Swipe Detection และ Cursor Dodge System
-

8. พื้นหลังด่านที่ 4

หน้าด่านที่ 4:

ฝึกหายใจ ดูแลสุขภาพ

Healthy Breathing Training

ให้ใช้ภาพพื้นหลัง:

Bg/Healthy.png

ข้อกำหนด:

- แสดงภาพเต็มพื้นที่ Gameplay
 - ใช้ **cover**
 - จัดกึ่งกลาง
 - ใช้ Overlay สีฟ้าหรือเขียวแบบโปร่งแสง
 - วงเป้าหมายและ Progress Ring ต้องเห็นชัด
 - ข้อความ “หายใจเข้าช้า ๆ” และ “หายใจออกช้า ๆ” ต้องอ่านง่าย
 - ลดความเคลื่อนไหวของพื้นหลังเพื่อไม่ให้เกิดการรบกวน
 - เมื่อเปิด Reduced Motion ให้ปิด Animation ของพื้นหลังทั้งหมด
 - หากภาพโหลดไม่ได้ ให้ใช้ Gradient โทนฟ้า-เขียวเดิม
 - ห้ามทำให้กิจกรรมกล่าวอ้างว่าสามารถตรวจจัดการหายใจจริง
-

9. พื้นหลังด่านที่ 5

หน้าด่านที่ 5:

อาหารดีมีประโยชน์

Healthy Food Basket

ให้ใช้ภาพพื้นหลัง:

Bg/FoodBasket.png

ข้อกำหนด:

- แสดงเต็มพื้นที่ Gameplay

- ใช้ **cover**
- จัดกึ่งกลาง
- เพิ่ม Overlay ที่ทำให้อาหารและตะกร้ามองเห็นชัด
- อาหารทุกชิ้นต้องแยกจากพื้นหลังได้ชัดเจน
- พื้นที่ตะกร้าต้องมีเส้นขอบหรือ Glow แสดง Drop Zone
- ห้ามให้พื้นหลังรับ Pointer Event
- ต้องไม่กระทบ Drag, Drop, Pinch, Release และ Hitbox
- หากภาพโหลดไม่ได้ ให้ใช้ Gradient โทนเขียว เหลือง หรือส้มเดิม

10. การจัดการพื้นหลังด้วย Asset Manifest

ให้เพิ่มหรือปรับ Asset Manifest กลาง เช่น:

```
const ASSETS = {
  backgrounds: {
    start: "Bg/Poster.png",
    level1: "Bg/Poster.png",
    level2: "Bg/Monsters.png",
    level3: "Bg/MegaBoss.png",
    level4: "Bg/Healthy.png",
    level5: "Bg/FoodBasket.png"
  }
};
```

ห้ามพิมพ์ Path เดียวกันซ้ำกระจายในหลายฟังก์ชัน

ให้สร้างฟังก์ชันกลางสำหรับเปลี่ยนพื้นหลัง เช่น:

```
setScreenBackground(screenElement, assetPath, fallbackClass)
```

หรือ:

```
applyLevelBackground(levelNumber)
```

ฟังก์ชันต้อง:

- ตรวจสอบ Asset ก่อนใช้
- มี Error Handling
- ใช้ Fallback เมื่อภาพไม่พบ
- ลบ Class หรือ Background ของหน้าก่อนหน้า
- ไม่สร้าง **** ที่แสดง Broken Image Icon
- ไม่โหลด Asset เดิมซ้ำโดยไม่จำเป็น

11. การป้องกันปัญหาพื้นหลังทับระบบเกม

ตรวจสอบ Layer ด้วย **z-index** ให้มีลำดับโดยประมาณดังนี้

1. Background Image
2. Background Overlay
3. Camera Video เฉพาะโหมด AR
4. Gameplay Canvas

5. Gameplay Objects และ Cursor
6. HUD
7. Toast
8. Modal และ Pause Menu

พื้นหลังและ Overlay ต้องใช้:

pointer-events: none;

ห้ามให้พื้นหลังขวางการคลิกแตะ ลาก หรือ Gesture ของผู้เล่น

12. Resource Cleanup

เมื่อเปลี่ยนหน้า เปลี่ยนด่าน หรือกลับหน้าแรก ให้ดำเนินการดังนี้

- ยกเลิก Speech Synthesis ที่ยังเล่นอยู่
 - ลบ Listener ชั่วคราวของเสียงต้อนรับ
 - รีเซ็ตตัวแปรเสียงของตัวละคร
 - ลบ Background Class ของหน้าก่อนหน้า
 - ไม่สร้าง Event Listener ซ้ำ
 - ไม่สร้าง Timeout ซ้ำ
 - ไม่สร้างเสียงซ้ำจาก Game Loop
 - ไม่โหลดภาพเดิมซ้ำทุก Frame
 - ไม่กระทบ MediaPipe Camera Loop
 - ไม่หยุดเสียงเอฟเฟกต์โดยไม่จำเป็น
-

13. Acceptance Criteria เพิ่มเติม

ก่อนส่งโค้ด ให้ตรวจสอบว่าระบบต่อไปนี้ทำงานจริง

หน้า Start Screen

- แสดงพื้นหลัง Bg/Poster.png
- พื้นหลังไม่บิดเบี้ยว
- ข้อความและปุ่มอ่านง่าย
- หลังโหลดระบบเสร็จ ให้เล่นเสียงต้อนรับ 1 ครั้ง
- หาก Browser ป้องกันเสียงอัตโนมัติ เสียงเล่นหลังการคลิกหรือแตะครั้งแรก
- เสียงไม่เล่นซ้ำจาก Resize หรือ Game Loop
- ปิดเสียงบรรยายแล้วต้องไม่เล่นเสียงต้อนรับ

หน้า Character Select

- ชื่อเมาส์ที่ Sky แล้วพูดชื่อและฉายาของ Sky
- ชื่อเมาส์ที่ Zee แล้วพูดชื่อและฉายาของ Zee
- ชื่อเมาส์ที่ Bubble แล้วพูดชื่อและฉายาของ Bubble
- แต่ละตัวละครแล้วเลือกตัวละครและเล่นเสียง
- ใช้ Keyboard Focus แล้วมีเสียงบรรยาย
- เสียงไม่ซ้อนกัน
- เสียงไม่เล่นซ้ำถี่เกินไป
- Event Listener ไม่ถูกเพิ่มซ้ำเมื่อกลับเข้าหน้านี้หลายครั้ง

พื้นหลังแต่ละด่าน

- ด่าน 1 ใช้ Bg/Poster.png

- ด้าน 2 ใช้ Bg/Monsters.png
- ด้าน 3 ใช้ Bg/MegaBoss.png
- ด้าน 4 ใช้ Bg/Healthy.png
- ด้าน 5 ใช้ Bg/FoodBasket.png
- หากภาพใดไม่พบ เกมยังเล่นได้
- ไม่มี Broken Image Icon
- Background ไม่ขวาง Mouse, Touch หรือ AR
- HUD อ่านง่ายทุกด้าน
- Cursor และ Hitbox ยิงตรงกับวัตถุ
- Responsive ทั้ง Portrait และ Landscape
- ไม่มี SyntaxError
- ไม่มี ReferenceError
- ไม่มี Listener ซ้ำ
- ไม่มีเสียงซ้อน
- ไม่มีการเปลี่ยน Gameplay Logic เดิม

14. รูปแบบการส่งมอบ

ให้ตอบกลับด้วยไฟล์ [index.html](#) ฉบับปรับปรุงที่สมบูรณ์ โดยมีเงื่อนไขดังนี้

1. ใช้โค้ดเดิมเป็นฐาน
2. แก้เฉพาะส่วนที่เกี่ยวข้องและส่วนที่จำเป็น
3. ห้ามตัดระบบเกมเดิม
4. ห้ามลดจำนวนด้าน
5. ห้ามสร้าง Pseudocode
6. ห้ามมี TODO
7. ฟังก์ชันทุกตัวต้องมี Logic ใช้งานจริง
8. ตรวจสอบ Path ของ Asset ทุกไฟล์
9. ตรวจสอบ Syntax, ID, Function, Event Listener และ Game Flow
10. ต้องพร้อมใช้งานผ่าน GitHub Pages
11. หากภาพหรือ Speech Synthesis ใช้ไม่ได้ เกมต้องยังเล่นได้ด้วย Mouse และ Touch ได้ครบทั้ง 5 ด้าน
12. ให้ความสำคัญกับความเสถียรและการใช้งานจริงมากกว่าเอฟเฟกต์ตกแต่ง

คำสั่งที่ 8

แก้ปัญหากล่อง AR เปิดซ้ำเมื่อเปิด-ปิดเสียงบรรยาย

ให้นำไฟล์ [index.html](#) เวอร์ชันล่าสุดมาปรับปรุงเพื่อแก้ปัญหา:

Failed to acquire camera feed:

NotReadableError: Could not start video source

ปัญหานี้เกิดขึ้นเมื่อระบบพยายามเปิดกล่องซ้ำ ขณะที่กล่องเดิมยังทำงานอยู่หรือยังไม่ได้ปล่อย [MediaStream Track](#) อย่างสมบูรณ์ ให้แก้ไขเฉพาะระบบเสียงบรรยาย ระบบกล่อง AR และ Resource Cleanup ที่เกี่ยวข้อง โดยห้ามสร้างเกมใหม่ทั้งหมด ห้ามลบ Gameplay เดิม และห้ามเปลี่ยนกติกาของทั้ง 5 ด้าน

1. แยกระบบเสียงบรรยายออกจากระบบกล่องอย่างเด็ดขาด

ฟังก์ชันเปิด-ปิดเสียงบรรยายต้องควบคุมเฉพาะ [SpeechSynthesis API](#) เท่านั้น

เมื่อผู้เล่นกดเปิดหรือปิดเสียงบรรยาย ห้ามเรียกฟังก์ชันต่อไปนี้:

```
startARControls()
stopARControls()
setControlMode()
stopAllInputSystems()
requestCamera()
showScreen()
```

ให้แก้ไข [toggleSetting\('narration'\)](#) เป็นแนวทางดังนี้:

```
function toggleSetting(type) {
  if (type === 'narration') {
    gameState.narrationEnabled = !gameState.narrationEnabled;

    if (!gameState.narrationEnabled) {
      stopSpeech();
    }

    updateSettingsUI();
    saveProgress();
    return;
  }

  // การตั้งค่าอื่นให้ใช้ Logic เดิม
}
```

ข้อกำหนด:

- การปิดเสียงบรรยายต้องเรียกเฉพาะ [stopSpeech\(\)](#)
 - การเปิดเสียงบรรยายไม่ต้องเปิดกล่องใหม่
 - การเปิดเสียงบรรยายไม่ต้องเปลี่ยนหน้าจอ
 - การเปิดเสียงบรรยายไม่ต้องเล่นข้อความเดิมอัตโนมัติ
 - การเปิด-ปิดเสียงบรรยายต้องไม่เปลี่ยน [gameState.controlMode](#)
 - การเปิด-ปิดเสียงบรรยายต้องไม่เปลี่ยน [isARActive](#)
 - บันทึกค่าลง LocalStorage ทันที
-

2. เพิ่มสถานะวงจรชีวิตของกล้อง AR

ให้เพิ่มตัวแปรกลางสำหรับควบคุมสถานะกล้อง:

```
const arLifecycle = {  
  status: 'idle',  
  stream: null,  
  startPromise: null,  
  stopPromise: null,  
  processingFrameId: null,  
  sessionToken: 0,  
  lastError: null  
};
```

ค่าของ **status** อนุญาตให้มีเพียง:

idle
starting
active
stopping
error

ห้ามใช้ Boolean หลายตัวที่อาจขัดแย้งกัน เช่น:

isOpeningCamera
cameraStarted
cameraLoading
cameraReady

ให้ใช้ **arLifecycle.status** เป็นแหล่งข้อมูลหลักเพียงแห่งเดียว

3. ห้ามเปิดกล้องซ้ำ

ให้แก่ **startARControls()** เพื่อป้องกันการเรียกซ้ำ

เงื่อนไข:

1. หากสถานะเป็น **starting** ให้คืนค่า Promise เดิม
2. หากสถานะเป็น **active** และ Stream ยังใช้งานได้ ให้ไม่เปิดกล้องใหม่
3. หากสถานะเป็น **stopping** ให้รอการหยุดกล้องเสร็จก่อน
4. ปุ่มลองใหม่หรือปุ่มยืนยันต้องไม่สามารถเรียกกล้องซ้อนกัน
5. ใช้ Promise กลางเพียงตัวเดียว

โครงสร้างที่ต้องใช้:

```
async function startARControls() {  
  if (arLifecycle.status === 'starting' && arLifecycle.startPromise) {  
    return arLifecycle.startPromise;  
  }  
  
  if (arLifecycle.status === 'active' && isCameraStreamAlive()) {  
    updateARStatus('กล้องพร้อมใช้งาน', 'success');  
    return true;  
  }  
}
```

```
arLifecycle.startPromise = startARSession();
```

```
try {
  return await arLifecycle.startPromise;
} finally {
  arLifecycle.startPromise = null;
}
}
```

4. ใช้ `getUserMedia` โดยตรง

ให้ยกเลิกการใช้ `new Camera()` จาก `camera_utils` สำหรับเปิดกล้อง เพราะระบบดังกล่าวอาจเก็บ Stream ภายใน และทำให้ปล่อยกล้องได้ไม่สมบูรณ์

ให้ใช้:

```
navigator.mediaDevices.getUserMedia()
```

โดยตรง

ให้สร้างฟังก์ชัน:

```
async function requestCameraStream()
```

ตัวอย่างเงื่อนไข:

```
const preferredConstraints = {
  audio: false,
  video: {
    facingMode: { ideal: 'user' },
    width: { ideal: 640 },
    height: { ideal: 480 }
  }
};
```

หาก Constraint แรกไม่สำเร็จ ให้ลองอีกครั้งด้วย:

```
{
  audio: false,
  video: true
}
```

แต่ห้าม Retry อัตโนมัติมากกว่าหนึ่งครั้ง

5. ขั้นตอนเปิดกล้องที่ถูกต้อง

ให้สร้างฟังก์ชัน:

```
async function startARSession()
```

ลำดับการทำงานต้องเป็นดังนี้:

1. กำหนด `arLifecycle.status = 'starting'`
2. Disable ปุ่มเริ่ม AR และปุ่มลองใหม่
3. ตรวจสอบ Secure Context
4. ตรวจสอบ `navigator.mediaDevices`
5. ตรวจสอบ `navigator.mediaDevices.getUserMedia`
6. โหลด MediaPipe ด้วย `loadMediaPipeOnce()`

7. เรียก `releaseCameraResources()` เพื่อเคลียร์ Stream เก่าที่อาจค้าง
8. รอประมาณ 250–400 มิลลิวินาที
9. เรียก `requestCameraStream()`
10. เก็บ Stream ลง `arLifecycle.stream`
11. กำหนด `arVideoElement.srcObject = stream`
12. รอ `loadedmetadata`
13. เรียก `await arVideoElement.play()`
14. เริ่ม MediaPipe Processing Loop
15. กำหนด `arLifecycle.status = 'active'`
16. กำหนด `gameState.controlMode = 'ar'` เฉพาะเมื่อเปิดกล้องสำเร็จ
17. เปิดปุ่ม “เริ่มภารกิจ”
18. แสดงข้อความ “กล้องพร้อมใช้งาน”

ห้ามกำหนด `controlMode = 'ar'` ก่อนเปิดกล้องสำเร็จ

6. ตรวจสอบว่า Stream ยังทำงานอยู่หรือไม่

ให้เพิ่มฟังก์ชัน:

```
function isCameraStreamAlive() {  
  const stream = arLifecycle.stream;  
  
  if (!stream) return false;  
  
  const videoTracks = stream.getVideoTracks();  
  
  return videoTracks.some(track =>  
    track.readyState === 'live' &&  
    track.enabled === true  
  );  
}
```

หาก Stream ยังทำงานอยู่ ห้ามเรียก `getUserMedia()` ใหม่

7. MediaPipe Processing Loop

ให้ใช้ `requestAnimationFrame` สำหรับส่งภาพจาก Video ไปยัง MediaPipe ให้สร้าง:

`async function processARFrame(sessionToken)`

ข้อกำหนด:

- ประมวลผลเฉพาะเมื่อ `arLifecycle.status === 'active'`
- ตรวจสอบว่า `sessionToken` ตรงกับ Session ปัจจุบัน
- ตรวจสอบว่า `arVideoElement.readyState >= 2`
- ห้ามสร้าง Processing Loop ซ้อน
- เก็บ ID ไว้ใน `arLifecycle.processingFrameId`
- เมื่อหยุด AR ต้องยกเลิก Frame ด้วย `cancelAnimationFrame()`

โครงสร้าง:

```
async function processARFrame(sessionToken) {  
  if (  
    arLifecycle.status !== 'active' ||
```

```

    sessionToken !== arLifecycle.sessionToken
  ){
    return;
  }

  if (
    arHands &&
    arVideoElement.readyState >= HTMLMediaElement.HAVE_CURRENT_DATA
  ){
    try {
      await arHands.send({ image: arVideoElement });
    } catch (error) {
      console.warn('MediaPipe frame error:', error);
    }
  }

  arLifecycle.processingFrameId =
    requestAnimationFrame(() => processARFrame(sessionToken));
}

```

8. ปล่อยกลิ้งอย่างสมบูรณ์

ให้สร้างฟังก์ชันกลาง:

```
async function releaseCameraResources()
```

ฟังก์ชันนี้ต้องทำงานครบดังนี้:

```

async function releaseCameraResources() {
  if (arLifecycle.status === 'stopping' && arLifecycle.stopPromise) {
    return arLifecycle.stopPromise;
  }
}

```

```

arLifecycle.stopPromise = (async () => {
  arLifecycle.status = 'stopping';
  arLifecycle.sessionToken++;

```

```

  if (arLifecycle.processingFrameId !== null) {
    cancelAnimationFrame(arLifecycle.processingFrameId);
    arLifecycle.processingFrameId = null;
  }

```

```
const streams = new Set();
```

```

if (arLifecycle.stream) {
  streams.add(arLifecycle.stream);
}

```

```
if (arVideoElement.srcObject instanceof MediaStream) {
```

```
streams.add(arVideoElement.srcObject);  
}
```

```
streams.forEach(stream => {  
  stream.getTracks().forEach(track => {  
    try {  
      track.stop();  
      track.enabled = false;  
    } catch (error) {  
      console.warn('Unable to stop camera track:', error);  
    }  
  });  
});
```

```
arLifecycle.stream = null;
```

```
try {  
  arVideoElement.pause();  
} catch (error) {  
  console.warn(error);  
}
```

```
arVideoElement.srcObject = null;  
arVideoElement.removeAttribute('src');  
arVideoElement.load();
```

```
inputState.active = false;  
inputState.pressed = false;  
inputState.released = false;
```

```
arState.handDetected = false;  
arState.isPinching = false;  
arState.wasPinching = false;  
arState.isOpen = false;
```

```
await new Promise(resolve => setTimeout(resolve, 300));
```

```
arLifecycle.status = 'idle';  
})();
```

```
try {  
  await arLifecycle.stopPromise;  
} finally {  
  arLifecycle.stopPromise = null;  
}  
}
```

9. แก้ stopARControls()

ให้เปลี่ยน `stopARControls()` เป็น Async Function:

```
async function stopARControls() {  
    await releaseCameraResources();  
}
```

ห้ามใช้เพียง:

```
arCamera.stop();
```

เพราะไม่รับประกันว่า Browser จะปล่อย `MediaStream Track` ทั้งหมด

10. แก้ stopAllInputSystems()

`stopAllInputSystems()` ต้องไม่หยุดกล้องทุกครั้งโดยไม่ตรวจสอบเหตุผล

ให้แยกระหว่าง:

- ปิด Input ของ Mouse
- ปิด Input ของ Touch
- ปิด Input ของ AR

สร้างฟังก์ชัน:

```
function stopMouseControls()  
function stopTouchControls()  
async function stopARControls()
```

และ:

```
async function stopAllInputSystems(options = {}) {  
    stopMouseControls();  
    stopTouchControls();  
  
    if (options.stopCamera === true) {  
        await stopARControls();  
    }  
  
    resetInputState();  
}
```

ห้ามให้การเปิด Settings หรือการเปิด-ปิดเสียงบรรยายเรียก:

```
stopAllInputSystems({ stopCamera: true })
```

11. ห้าม cleanupCurrentScreen() ปิดกล้องโดยอัตโนมัติทุกหน้า

ฟังก์ชัน `cleanupCurrentScreen()` ปัจจุบันต้องไม่เรียก `stopARControls()` ทุกครั้งที่เปลี่ยนหน้าจอ เพราะผู้เล่นอาจเปลี่ยนระหว่าง:

```
screen-ar-check  
screen-map  
screen-game  
modal-settings  
modal-pause
```

ทั้งที่ยังใช้โหมด AR อยู่

ให้หยุดกล้องเฉพาะกรณี:

- เปลี่ยนจาก AR ไป Mouse
- เปลี่ยนจาก AR ไป Touch
- กลับหน้า Start
- ออกจากเกมไปหน้า Character Select
- Page ถูกซ่อนและต้องหยุดกล้องตามนโยบาย
- เกิด Error ที่ทำให้ AR ใช้งานไม่ได้
- ผู้เล่นกดปิด AR อย่างชัดเจน

การเปิดหรือปิด Modal Settings ห้ามหยุดกล้อง

12. แก้การเปลี่ยนโหมดควบคุม

ก่อนเปลี่ยนไป AR ให้เก็บโหมดเดิม:

```
const previousMode = gameState.controlMode;
```

เมื่อเปิด AR สำเร็จจึงกำหนด:

```
gameState.controlMode = 'ar';
```

หากเปิดกล้องไม่สำเร็จ:

```
gameState.controlMode = previousMode;
```

และต้อง:

- ไม่รีเซ็ตเวลา
 - ไม่รีเซ็ต HP
 - ไม่รีเซ็ตคะแนน
 - ไม่รีเซ็ตด่าน
 - ไม่เปลี่ยน `gameState.status`
 - ไม่ออกจาก Pause โดยอัตโนมัติ
-

13. แก้ปุ่ม Retry AR

ฟังก์ชัน `retryAR()` ต้อง:

1. Disable ปุ่ม Retry ทันที
2. เรียก `await releaseCameraResources()`
3. รอประมาณ 300 มิลลิวินาที
4. เรียก `await startARControls()`
5. Enable ปุ่ม Retry เมื่อเสร็จ
6. ห้ามเปิดกล้องซ้ำหากมี Promise เดิมกำลังทำงาน
7. ห้ามใช้ Loop Retry อัตโนมัติ

ตัวอย่าง:

```
async function retryAR() {  
  const retryButton = document.getElementById('btn-ar-retry');  
  
  if (retryButton) retryButton.disabled = true;  
  
  try {  
    await releaseCameraResources();
```



```

    await startARControls();
  } finally {
    if (retryButton) retryButton.disabled = false;
  }
}

```

14. Error Handling ตามประเภทข้อผิดพลาด

ให้สร้างฟังก์ชัน:

```
function getCameraErrorMessage(error)
```

ข้อความต้องแยกตาม **error.name**

```

function getCameraErrorMessage(error) {
  switch (error?.name) {
    case 'NotAllowedError':
    case 'PermissionDeniedError':
      return 'ไม่ได้รับอนุญาตให้ใช้กล้อง กรุณาอนุญาตสิทธิ์กล้องในการตั้งค่า Browser';

    case 'NotFoundError':
    case 'DevicesNotFoundError':
      return 'ไม่พบกล้องที่สามารถใช้งานได้ กรุณาตรวจสอบอุปกรณ์กล้อง';

    case 'NotReadableError':
    case 'TrackStartError':
      return 'ไม่สามารถเริ่มกล้องได้ กล้องอาจถูกใช้งานโดยโปรแกรมหรือเว็บไซต์อื่น กรุณาปิด Camera, Zoom, Meet หรือแท็บอื่น แล้วลองใหม่';

    case 'OverconstrainedError':
    case 'ConstraintNotSatisfiedError':
      return 'กล้องไม่รองรับค่าความละเอียดที่กำหนด ระบบจะลองใช้ค่ามาตรฐาน';

    case 'AbortError':
      return 'การเปิดกล้องถูกยกเลิก กรุณาลองใหม่อีกครั้ง';

    case 'SecurityError':
      return 'Browser ไม่อนุญาตให้เปิดกล้อง กรุณาเปิดเกมผ่าน HTTPS หรือ localhost';

    default:
      return 'ไม่สามารถเปิดกล้องได้ กรุณาตรวจสอบสิทธิ์กล้องและลองใหม่';
  }
}

```

ให้แสดงข้อความผ่าน Custom Modal หรือ Toast เท่านั้น

ห้ามใช้:

```
alert()
```

```
confirm()
```

```
prompt()
```

15. การตรวจ Secure Context

ก่อนเปิด AR ให้ตรวจสอบ:

`window.isSecureContext`

และ:

`navigator.mediaDevices?.getUserMedia`

หากไม่พร้อม ให้ปิดปุ่ม AR และแสดงข้อความ:

โหมดกล้อง AR ต้องเปิดผ่าน HTTPS หรือ localhost กรุณาใช้ GitHub Pages หรือเลือกโหมดเมาส์หรือสัมผัส

ห้ามพยายามเรียกกล้องต่อ

16. Page Visibility API

เมื่อหน้าเว็บถูกซ่อน:

`document.hidden === true`

ให้:

- Pause เกม
- หยุด MediaPipe Processing Loop
- ปลด Stream ของกล้อง
- จำไว้ว่าเดิมผู้เล่นใช้โหมด AR

เมื่อกลับมาที่หน้าเว็บ:

- ห้ามเปิดกล้องอัตโนมัติ
 - แสดงข้อความ “กล้องถูกหยุดชั่วคราว กรุณากดเริ่มกล้องอีกครั้ง”
 - ให้ผู้เล่นกดปุ่ม Resume หรือเริ่มกล้องเอง
 - ห้ามรีเซ็ตเวลา HP คะแนน หรือด่าน
-

17. ป้องกัน Event Listener ซ้ำ

Global Listener เช่น:

`pointerdown`

`visibilitychange`

`resize`

`orientationchange`

ต้องเพิ่มเพียงครั้งเดียวใน `initGame()`

ห้ามเพิ่ม Listener ซ้ำทุกครั้งที่เปิดหน้า AR หรือ Settings

ให้ใช้ Named Function แทน Anonymous Function เมื่อจำเป็นต้องลบ Listener

18. ปุ่มเปิด-ปิดเสียงบรรยายระหว่างใช้ AR

เมื่อผู้เล่นกำลังใช้กล้อง AR และเปิดหน้า Settings:

- ภาพกล้องต้องยังแสดงต่อ
- Stream ต้องยังมีสถานะ `live`
- MediaPipe ต้องยังทำงาน
- การปิดเสียงบรรยายต้องหยุดเฉพาะคำพูด
- การเปิดเสียงบรรยายต้องไม่เรียก `getUserMedia()`

- การปิด Settings ต้องไม่เรียก `startARControls()`
- ห้ามสร้างกล่อง Session ใหม่
- ห้ามเปลี่ยน `arLifecycle.sessionToken`

19. Acceptance Criteria

ก่อนส่งโค้ด ให้ทดสอบกรณีต่อไปนี้

การเปิดกล่อง

- เลือก AR แล้วเปิดกล่องได้
- กดยืนยัน AR ติดต่อกันหลายครั้ง แต่ `getUserMedia()` ถูกเรียกเพียงครั้งเดียว
- เมื่อกล่องเปิดแล้วเรียก `startARControls()` ซ้ำ ระบบไม่เปิดกล่องใหม่
- ไม่มี Video Stream ซ้อนกัน
- ไม่มี MediaPipe Processing Loop ซ้อนกัน

การเปิด-ปิดเสียงบรรยาย

- เปิดกล่อง AR แล้วเปิด Settings
- กดปิดเสียงบรรยาย
- กล่องยังทำงาน
- ภาพ Video ไม่ดับ
- MediaPipe ยังตรวจจับมือได้
- ไม่มี `NotReadableError`
- กดเปิดเสียงบรรยายอีกครั้ง
- กล่องยังทำงาน
- ไม่มีการเรียก `getUserMedia()` ใหม่
- การปิดเสียงบรรยายเรียกเฉพาะ `stopSpeech()`

การเปลี่ยนโหมด

- เปลี่ยนจาก AR ไป Mouse แล้ว Camera Track ทุกตัวเป็น `ended`
- `arVideoElement.srcObject` ต้องเป็น `null`
- เปลี่ยนจาก Mouse กลับไป AR แล้วกล่องเปิดใหม่ได้
- เปลี่ยนจาก Touch กลับไป AR แล้วกล่องเปิดใหม่ได้
- คะแนน HP เวลา และสถานะด้านไม่ถูกรีเซ็ต

การ Retry

- เมื่อกล่องถูกโปรแกรมอื่นใช้งาน แสดงข้อความที่เข้าใจง่าย
- ปุ่ม Retry ไม่เปิดกล่องซ้ำ
- เมื่อปิดโปรแกรมที่ใช้กล่องแล้วกด Retry สามารถเปิดกล่องได้
- ไม่มี Retry Loop อัตโนมัติ

การออกจาก AR

- กลับหน้า Start แล้วกล่องดับ
- ปิดแท็บหรือซ่อนหน้าแล้วกล่องถูกปล่อย
- ไม่มี Camera Indicator ค้างหลังออกจาก AR
- ไม่มี `MediaStream Track` ที่ยังมีสถานะ `live`
- ไม่มี Animation Frame ของ AR ค้างอยู่

Console

ต้องไม่มี:

`NotReadableError`

`Failed to acquire camera feed`

`Unhandled Promise Rejection`

`Cannot read properties of null`

`Camera already started`

ข้อผิดพลาดของกล่องที่ควบคุมไม่ได้ต้องถูกจับด้วย `try...catch` และแสดงผ่าน UI ภายในเกม

20. รูปแบบการส่งมอบ

ให้ตอบกลับด้วยโค้ด `index.html` ฉบับสมบูรณ์ โดยมีเงื่อนไขดังนี้:

1. ใช้ไฟล์เดิมเป็นฐาน
2. ห้ามสร้างเกมใหม่ทั้งหมด
3. ห้ามลบ Gameplay เดิม
4. ห้ามลดจำนวนด่าน
5. ห้ามเปลี่ยนกติกาคะแนน HP เวลา หรือเงื่อนไขผ่านด่าน
6. แก้เฉพาะ Speech System, AR Lifecycle, Camera Stream และ Cleanup ที่เกี่ยวข้อง
7. ห้ามมี TODO
8. ห้ามมี Pseudocode
9. ฟังก์ชันทุกตัวต้องมี Logic ใช้งานจริง
10. ต้องใช้ `getUserMedia()` โดยตรง
11. ต้องปล่อย `MediaStream Track` ทุกตัวเมื่อออกจาก AR
12. ต้องไม่เปิดกล่องใหม่เมื่อเปิด-ปิดเสียงบรรยาย
13. ต้องไม่สร้าง AR Processing Loop ซ้ำ
14. ต้องมี Error Handling สำหรับ `NotReadableError`
15. ต้องทำงานผ่าน GitHub Pages
16. Mouse และ Touch ต้องยังเล่นได้ครบทั้ง 5 ด้าน
17. หากกล่องหรือ MediaPipe ล้มเหลว เกมต้องไม่ค้าง
18. ตรวจสอบ SyntaxError, ReferenceError, Event Listener ซ้ำ, Timer ซ้ำ, Stream ซ้ำ และ Game Loop ซ้อนก่อนส่ง
19. ตอบด้วย Code Block ภาษา HTML เพียง Code Block เดียว
20. ห้ามอธิบายนอก Code Block

คำสั่งที่ 9

ปรับปรุงด่านที่ 5 “อาหารดีมีประโยชน์” และระบบ AR ติดตามนิ้วมือทั้ง 5 นิ้ว

ให้นำไฟล์ [index.html](#) เวอร์ชันล่าสุดที่แก้ปัญหากล้อง AR และ `NotReadableError` ตามคำสั่งที่ 8 แล้ว มาปรับปรุงเฉพาะด่านที่ 5

ห้ามสร้างเกมใหม่ทั้งหมด ห้ามลบระบบเดิม ห้ามเปลี่ยน Gameplay ของด่านที่ 1–4 และห้ามทำให้ระบบ Mouse, Touch, AR, คะแนน, LocalStorage, Pause หรือ Mission Map ที่มีอยู่เดิมเสียหาย

1. ขอบเขตการปรับปรุง

ให้ปรับปรุงเฉพาะส่วนต่อไปนี้:

- ข้อมูลด่านที่ 5
- เวลาเล่นด่านที่ 5
- รายการอาหาร
- การจำแนกอาหารครบ 5 หมู่
- Asset Manifest ของอาหาร
- ระบบสุ่มและแสดงอาหาร
- ระบบลากอาหารลงตะกร้า
- ระบบตรวจคำตอบ
- ระบบคะแนน
- ระบบ Progress อาหารครบ 5 หมู่
- ระบบ AR สำหรับจับและลากอาหาร
- การแสดงตำแหน่งนิ้วมือทั้ง 5 นิ้ว
- Result Screen ของด่านที่ 5
- Acceptance Criteria ของด่านที่ 5

ห้ามแก้ระบบเปิดกล้องที่ปรับปรุงแล้วจากคำสั่งที่ 8 ยกเว้นส่วนส่งข้อมูล Landmark ไปยัง Input State ของด่านที่ 5

2. ข้อมูลด่านที่ 5

กำหนดข้อมูลด่านดังนี้:

ชื่อภาษาไทย: ด่านที่ 5 อาหารดีมีประโยชน์

ชื่อภาษาอังกฤษ: Healthy Food Basket

เวลา: 45 วินาที

วัตถุประสงค์: เลือกอาหารที่มีประโยชน์และเก็บอาหารให้ครบทั้ง 5 หมู่

พื้นหลัง: Bg/FoodBasket.png

ให้แก่ `LEVELS_DATA` ของด่านที่ 5 เป็นเวลา 45 วินาที

ข้อความคำแนะนำบน HUD:

หยิบอาหารที่มีประโยชน์จากแต่ละหมู่ แล้วลากลงตะกร้าให้ครบ 5 หมู่

ข้อความรอง:

ต้องเก็บอาหารจากแต่ละหมู่อย่างน้อย 1 ชิ้น

ต้องแสดงข้อความ:

ข้อมูลอาหารในเกมจัดทำขึ้นเพื่อการเรียนรู้เรื่องอาหาร 5 หมู่

3. โครงสร้างข้อมูลอาหาร

ให้สร้างข้อมูลอาหารส่วนกลางในรูปแบบ Object หรือ Array โดยอาหารแต่ละรายการต้องมีข้อมูลอย่างน้อยดังนี้:

```
{  
  id: "meat",  
  name: "เนื้อ",  
  groupId: 1,  
  groupName: "โปรตีน",  
  category: "healthy",  
  asset: "Food/Meat.png",  
  fallback: "🍖"  
}
```

ห้ามเขียนเงื่อนไขตรวจสอบชื่ออาหารแบบกระจายอยู่ในหลายฟังก์ชัน
ให้ใช้ข้อมูลอาหารส่วนกลางเพียงชุดเดียว เช่น:
`const LEVEL5_FOODS = [];`

4. อาหารหมู่ที่ 1: โปรตีน

ชื่อหมู่:

หมู่ 1 โปรตีน

เนื้อสัตว์ ไข่ นม และถั่ว

รายการอาหาร:

เนื้อ

Asset Path: Food/Meat.png

เนื้อปลา

Asset Path: Food/Fish.png

ไข่

Asset Path: Food/Eggs.png

นม

Asset Path: Food/Milk.png

ถั่วลิสง

Asset Path: Food/Peanuts.png

กำหนด:

groupId: 1

groupName: "โปรตีน"

category: "healthy"

5. อาหารหมู่ที่ 2: คาร์โบไฮเดรต

ชื่อหมู่:

หมู่ 2 คาร์โบไฮเดรต

ข้าว แป้ง น้ำตาล เผือก และมัน

รายการอาหาร:

ข้าว

Asset Path: Food/Rice.png

เผือก

Asset Path: Food/Taro.png

มันเทศ

Asset Path: Food/Yam.png

น้ำตาล

Asset Path: Food/Sugar.png

กำหนด:

groupId: 2

groupName: "คาร์โบไฮเดรต"

category: "healthy"

6. อาหารหมู่ที่ 3: เกลือแร่หรือแร่ธาตุ

ชื่อหมู่:

หมู่ 3 เกลือแร่หรือแร่ธาตุ

พืชผัก

รายการอาหาร:

มะเขือเทศ

Asset Path: Food/Tomatoes.png

บรอกโคลี

Asset Path: Food/Broccoli.png

กะหล่ำปลี

Asset Path: Food/Cabbage.png

แครอท

Asset Path: Food/Carrot.png

ถั่วฝักยาว

Asset Path: Food/Green-beans.png

กำหนด:

groupId: 3

groupName: "เกลือแร่หรือแร่ธาตุ"

category: "healthy"

7. อาหารหมู่ที่ 4: วิตามิน

ชื่อหมู่:

หมู่ 4 วิตามิน

ผลไม้

รายการอาหาร:

ส้ม

Asset Path: Food/Orange.png

แอปเปิล

Asset Path: Food/Apple.png

แตงโม

Asset Path: Food/Watermelon.png

กล้วย

Asset Path: Food/Banana.png

ฝรั่ง

Asset Path: Food/Guava.png

กำหนด:

groupId: 4

groupName: "วิตามิน"

category: "healthy"

8. อาหารหมู่ที่ 5: ไขมัน

ชื่อหมู่:

หมู่ 5 ไขมัน

ไขมันจากพืชและสัตว์

รายการอาหาร:

น้ำมันปาล์ม

Asset Path: Food/Palm-oil.png

น้ำมันถั่วเหลือง

Asset Path: Food/Soybean-oil.png

เนย

Asset Path: Food/Butter.png

ชีส

Asset Path: Food/Cheese.png

กำหนด:

groupId: 5

groupName: "ไขมัน"

category: "healthy"

ให้ใช้การจัดหมู่อาหารตามข้อมูลที่กำหนดในเกมนี้อย่างสม่ำเสมอทุกจุด

9. อาหารที่ควรรับประทานแต่น้อย

รายการอาหาร:

น้ำอัดลม

Asset Path: Food/Soft-drinks.png

ลูกอม

Asset Path: Food/Candy.png

มันฝรั่งทอด

Asset Path: Food/Potato-chips.png

อาหารทอด

Asset Path: Food/Fried-food.png

ขนมหวาน

Asset Path: Food/Desserts.png

กำหนด:

groupId: null

groupName: "อาหารที่ควรรับประทานแต่น้อย"

category: "limited"

ห้ามกำหนดอาหารกลุ่มนี้เป็นอาหารอันตรายหรืออาหารต้องห้าม ให้ใช้ถ้อยคำว่า:
ควรรับประทานแต่น้อย

10. Asset Manifest

ให้เพิ่ม Asset Path ทั้งหมดลงใน Asset Manifest กลาง

ตัวอย่างโครงสร้าง:

```
const ASSETS = {  
  foods: {  
    meat: "Food/Meat.png",  
    fish: "Food/Fish.png",  
    eggs: "Food/Eggs.png",  
    milk: "Food/Milk.png",  
    peanuts: "Food/Peanuts.png",  
  
    rice: "Food/Rice.png",  
    taro: "Food/Taro.png",  
    yam: "Food/Yam.png",  
    sugar: "Food/Sugar.png",  
  
    tomatoes: "Food/Tomatoes.png",  
    broccoli: "Food/Broccoli.png",  
    cabbage: "Food/Cabbage.png",  
    carrot: "Food/Carrot.png",  
    greenBeans: "Food/Green-beans.png",  
  
    orange: "Food/Orange.png",  
    apple: "Food/Apple.png",  
    watermelon: "Food/Watermelon.png",  
    banana: "Food/Banana.png",  
    guava: "Food/Guava.png",  
  
    palmOil: "Food/Palm-oil.png",  
    soybeanOil: "Food/Soybean-oil.png",  
    butter: "Food/Butter.png",  
    cheese: "Food/Cheese.png",  
  
    softDrinks: "Food/Soft-drinks.png",  
    candy: "Food/Candy.png",  
    potatoChips: "Food/Potato-chips.png",  
    friedFood: "Food/Fried-food.png",  
    desserts: "Food/Desserts.png"  
  }  
};
```

ข้อกำหนด:

- ใช้ Relative Path
- ห้ามขึ้นต้นด้วย /
- ตรวจสอบตัวพิมพ์เล็กและใหญ่
- มี Emoji หรือ SVG Fallback

- ห้ามแสดง Broken Image Icon
 - หากภาพอาหารโหลดไม่สำเร็จ เกมต้องยังเล่นได้
 - ห้ามโหลดภาพใหม่ทุก Frame
-

11. สถานะเฉพาะด้านที่ 5

ให้สร้าง State สำหรับด้านที่ 5 เช่น:

```
const level5State = {  
  foods: [],  
  basket: null,  
  draggingFood: null,  
  collectedGroups: new Set(),  
  collectedFoodIds: new Set(),  
  correctCount: 0,  
  errorCount: 0,  
  score: 0,  
  spawnQueue: [],  
  feedbackText: "",  
  feedbackTimer: 0  
};
```

เมื่อเริ่มหรือ Restart ด้านที่ 5 ต้องรีเซ็ต State นี้ทั้งหมด
ห้ามใช้ **Set** เดิมจากการเล่นครั้งก่อน

12. ระบบสุ่มอาหาร

เมื่อเริ่มด้าน ให้เตรียมอาหารที่สามารถทำให้ผู้เล่นผ่านด้านได้จริง

ระบบต้องรับประกันว่า:

- มีอาหารจากหมู่ที่ 1 ปรากฏอย่างน้อย 1 รายการ
- มีอาหารจากหมู่ที่ 2 ปรากฏอย่างน้อย 1 รายการ
- มีอาหารจากหมู่ที่ 3 ปรากฏอย่างน้อย 1 รายการ
- มีอาหารจากหมู่ที่ 4 ปรากฏอย่างน้อย 1 รายการ
- มีอาหารจากหมู่ที่ 5 ปรากฏอย่างน้อย 1 รายการ
- มีอาหารที่ควรรับประทานแต่น้อยผสมอยู่ด้วย
- ห้ามสุ่มจนไม่มีอาหารบางหมู่ให้เลือก
- ห้ามให้อาหารวางทับกันมากจนจับไม่ได้
- ห้ามวางอาหารทับ HUD หรือตะกร้า
- รองรับหน้าจอแนวตั้งและแนวนอน

ให้แสดงอาหารพร้อมกันประมาณ 8–12 ชิ้น ตามขนาดหน้าจอ

เมื่ออาหารชิ้นหนึ่งถูกนำออก ให้สุ่มอาหารชิ้นใหม่เข้ามาแทน โดยต้องคำนึงถึงหมู่ที่ผู้เล่นยังเก็บไม่ครบ
ให้เพิ่มโอกาสสุ่มอาหารจากหมู่ที่ยังไม่ครบ เพื่อป้องกันผู้เล่นติดอยู่จนหมดเวลา

13. Progress อาหารครบ 5 หมู่

ให้แสดง Progress บนหน้าจอเป็น 5 ช่อง ได้แก่:

หมู่ 1 โปรตีน

หมู่ 2 คาร์โบไฮเดรต

หมู่ 3 เกลือแร่หรือแร่ธาตุ

หมู่ 4 วิตามิน

หมู่ 5 ไขมัน

สถานะของแต่ละหมู่:

○ ยังไม่ได้เก็บ

✓ เก็บสำเร็จแล้ว

ห้ามใช้สีเพียงอย่างเดียว ต้องมีข้อความหรือเครื่องหมายกำกับ

ตัวอย่าง HUD:

อาหารครบ 5 หมู่: 3/5

✓ หมู่ 1 โปรตีน

✓ หมู่ 2 คาร์โบไฮเดรต

○ หมู่ 3 เกลือแร่หรือแร่ธาตุ

✓ หมู่ 4 วิตามิน

○ หมู่ 5 ไขมัน

Progress ต้องเพิ่มเฉพาะเมื่อผู้เล่นนำอาหารจากหมู่ที่ยังไม่เคยเก็บลงตะกร้า

การนำอาหารหมู่เดิมลงตะกร้าซ้ำ:

- ยังถือว่าเป็นอาหารที่มีประโยชน์
- ได้รับคะแนนตามกติกา
- แต่ห้ามเพิ่มจำนวนหมู่เกินหนึ่งครั้ง
- ห้ามทำให้ Progress เกิน 5/5

14. ตะกร้าอาหาร

ให้แสดงตะกร้าบริเวณด้านล่างของพื้นที่เล่น

Asset ของตะกร้าให้ใช้ของเดิม หากไม่มีให้ใช้:

Images/Basket.png

หรือ Emoji fallback:



ข้อกำหนด:

- ตะกร้ามี Hitbox ชัดเจน
- แสดงเส้นขอบหรือ Glow
- แสดงข้อความ “วางอาหารที่นี่”
- เมื่อกำลังลากอาหารผ่านตะกร้า ให้เปลี่ยน Glow
- ตะกร้าต้องอยู่หลังอาหารที่กำลังลาก แต่หน้า Background
- ห้ามให้ HUD บังตะกร้า
- รองรับ Resize และ Orientation Change

15. การควบคุม AR ด้วยนิ้วมือทั้ง 5 นิ้ว

ให้ใช้ MediaPipe Hands และข้อมูล Landmark จริง

ระบบต้องติดตามปลายนิ้วทั้ง 5 จุด ได้แก่:

Landmark 4 = ปลายนิ้วโป้ง

Landmark 8 = ปลายนิ้วชี้

Landmark 12 = ปลายนิ้วกลาง

Landmark 16 = ปลายนิ้วนาง

Landmark 20 = ปลายนิ้วก้อย

ให้แสดงจุดควบคุมหรือวงกลมขนาดเล็กบนปลายนิ้วทั้ง 5 นิ้ว เพื่อให้ผู้เล่นเห็นว่าระบบตรวจจับมือครบ
สี่หรือไอคอนของแต่ละนิ้วต้องมองเห็นง่าย แต่ห้ามบดบังอาหาร

การใช้ข้อมูลนิ้วทั้ง 5 มีวัตถุประสงค์เพื่อ:

- ตรวจสอบว่าระบบเห็นมือครบ
- เพิ่มความเสถียรของตำแหน่งมือ
- ลดการจับผิดเมื่อเห็นเพียงบางนิ้ว
- แสดง Feedback ให้ผู้เล่นเข้าใจตำแหน่งมือ
- ช่วยคำนวณขนาดและทิศทางของมือ

อย่างไรก็ตาม คำสั่งจับและปล่อยอาหารให้ใช้ **Pinch Gesture** ระหว่างนิ้วโป้งกับนิ้วชี้เป็นกลไกหลัก
ห้ามกล่าวอ้างว่าระบบตรวจจับแรงจับจริงของนิ้วทั้ง 5

16. Palm Center และ Hand Stability

ให้คำนวณจุดกึ่งกลางฝ่ามือจาก Landmark:

0, 5, 9, 13 และ 17

ให้ใช้ Palm Center ประกอบการตรวจสอบความเสถียรของมือ

ให้คำนวณ Hand Scale จากระยะระหว่าง Landmark 5 และ 17 หรือระยะจากข้อมือไป Landmark 9 เพื่อ
ปรับค่า Pinch Threshold ตามขนาดมือและระยะห่างจากกล้อง

ห้ามใช้ค่า Pixel คงที่เพียงอย่างเดียว เพราะมือที่อยู่ใกล้และไกลกล้องมีขนาดแตกต่างกัน

17. Pinch Gesture สำหรับจับอาหาร

ให้ใช้ Landmark:

Landmark 4 = ปลายนิ้วโป้ง

Landmark 8 = ปลายนิ้วชี้

คำนวณระยะห่างระหว่าง Landmark 4 และ Landmark 8

ให้ใช้ระยะที่หารด้วย Hand Scale แล้ว เพื่อให้ค่าเป็นสัดส่วนกับขนาดมือ

กำหนดค่า:

PINCH_THRESHOLD

RELEASE_THRESHOLD

โดย:

RELEASE_THRESHOLD ต้องมากกว่า PINCH_THRESHOLD

เพื่อสร้าง Hysteresis และป้องกันสถานะจับ-ปล่อยสลับเร็วเกินไป

ตัวอย่างหลักการ:

```
if (!isPinching && normalizedDistance < PINCH_THRESHOLD) {
```

```

    isPinching = true;
}

if (isPinching && normalizedDistance > RELEASE_THRESHOLD) {
    isPinching = false;
}

```

ต้องมี Gesture Debounce อย่างน้อย 2–4 Frame ก่อนยืนยันสถานะจับหรือปล่อย

18. Pinch Cursor

ให้ใช้ตำแหน่งกึ่งกลางระหว่าง Landmark 4 และ Landmark 8 เป็น **Pinch Cursor**

```

pinchX = (thumbX + indexX) / 2;
pinchY = (thumbY + indexY) / 2;

```

ต้องแปลงพิกัดด้วยฟังก์ชันกลาง:

```
mapVideoPointToCanvas()
```

โดยรองรับ:

- ภาพกล้อง Mirror
- Video แบบ **object-fit: cover**
- Canvas Resize
- Orientation Change
- Desktop
- Tablet
- Mobile

ให้ใช้ Lerp หรือ Low-pass Filter:

```

smoothedX = lerp(smoothedX, targetX, 0.25);
smoothedY = lerp(smoothedY, targetY, 0.25);

```

ค่าความลื่นไหลสามารถปรับได้ตามประสิทธิภาพอุปกรณ์ แต่ต้องไม่หน่วงมากจนลากอาหารไม่ทันมือ

19. ขั้นตอนจับอาหารด้วย AR

การจับอาหารต้องทำตามลำดับนี้:

1. ตรวจพบมือ
2. ตรวจพบ Landmark ทั้ง 21 จุด
3. คำนวณตำแหน่งปลายนิ้วทั้ง 5
4. คำนวณ Pinch Cursor
5. ผู้เล่นนำ Pinch Cursor ไปอยู่บนอาหาร
6. ผู้เล่นบีบนิ้วโป้งและนิ้วชี้เข้าหากัน
7. ตรวจ Pinch Gesture ต่อเนื่องตาม Debounce
8. ตรวจว่า Cursor อยู่ใน Hitbox ของอาหาร
9. ล็อกอาหารชิ้นนั้นเป็น **draggingFood**
10. บันทึกตำแหน่งเริ่มต้นของอาหาร
11. ลากอาหารตาม Pinch Cursor

12. เมื่อปล่อย Pinch ให้ตรวจตำแหน่งอาหาร
13. หากอยู่ในตะกร้า ให้ตรวจคำตอบ
14. หากอยู่นอกตะกร้า ให้อาหารกลับตำแหน่งเดิม

ห้ามจับอาหารหลายชิ้นพร้อมกัน

หากอาหารวางซ้อนกัน ให้เลือกชิ้นที่อยู่บนสุดหรือใกล้ Cursor มากที่สุด

20. การลากอาหารด้วย AR

ขณะจับอาหาร:

- ล็อกอาหารกับ Pinch Cursor
- ใช้ Lerp เพื่อให้ตำแหน่งลื่นไหล
- แสดง Glow รอบอาหาร
- ยกอาหารขึ้นเหนืออาหารชิ้นอื่นด้วยลำดับการวาด
- แสดงเส้นหรือวงเชื่อมจากนิ้วโป้งและนิ้วชี้ไปยังอาหาร
- ป้องกันอาหารหลุดออกนอก Canvas
- จำกัดตำแหน่งอาหารให้อยู่ในพื้นที่ Gameplay

หากระบบไม่พบมือชั่วคราวไม่เกินประมาณ 250–400 มิลลิวินาที ให้ค้างอาหารไว้ชั่วคราว

หากไม่พบมือนานกว่าระยะที่กำหนด:

- ยกเลิกการลาก
 - คินอาหารไปยังตำแหน่งเริ่มต้น
 - รีเซ็ต Pinch State
 - ไม่หักคะแนน
 - แสดงข้อความ “ไม่พบมือ กรุณายกมือให้อยู่ในกรอบ”
-

21. การปล่อยอาหารด้วย AR

ให้ถือว่าปล่อยอาหารเมื่อ:

- ระยะนิ้วโป้งและนิ้วชี้มากกว่า `RELEASE_THRESHOLD`
- ผ่าน Gesture Debounce
- หรือไม่พบมือเกินระยะเวลาที่กำหนด

ห้ามใช้ Fist เป็นคำสั่งจับ

ห้ามใช้ Open Hand เป็นคำสั่งปล่อย

ห้ามใช้ Landmark 8 เพียงจุดเดียวเป็นคำสั่งจับ

ห้ามปล่อยอาหารเพียงเพราะ Cursor เคลื่อนออกจาก Hitbox หลังจับแล้ว

22. การควบคุม Mouse

ให้คงการควบคุมดังนี้:

1. กดเมาส์ค้างบนอาหารเพื่อจับ
2. ลากเมาส์เพื่อย้ายอาหาร
3. ปล่อยเมาส์เพื่อวางอาหาร
4. หากปล่อยในตะกร้า ให้ตรวจคำตอบ
5. หากปล่อยนอกตะกร้า ให้อาหารกลับตำแหน่งเดิม

ต้องรองรับ:

mousedown

mousemove

mouseup

หรือ Pointer Events ที่เทียบเท่า
เมื่อเมาส์ออกนอกหน้าต่างขณะลาก ต้องยกเลิกการลากอย่างปลอดภัย

23. การควบคุม Touch

ให้คงการควบคุมดังนี้:

1. แตะค้างบนอาหารเพื่อจับ
2. ลากนิ้วเพื่อย้ายอาหาร
3. ปล่อนิ้วเพื่อวางอาหาร
4. หากปล่อยในตะกร้า ให้ตรวจคำตอบ
5. หากปล่อยนอกตะกร้า ให้อาหารกลับตำแหน่งเดิม

ให้ใช้ Pointer Events เป็นหลัก หากโครงสร้างเกมรองรับอยู่แล้ว
ต้อง:

- ป้องกัน Scroll เฉพาะพื้นที่ Gameplay
 - ป้องกัน Pull to Refresh ระหว่างลาก
 - รองรับ Pointer Capture
 - ปล่อย Pointer Capture เมื่อวางอาหาร
 - ไม่ปิดการเลื่อนในหน้าจออื่นของเกม
-

24. ระบบตรวจคำตอบ

ให้สร้างฟังก์ชันกลาง เช่น:

`handleLevel5FoodDrop(food)`

หากอาหารมี:

`category === "healthy"`

ให้ถือว่าเป็นอาหารที่มีประโยชน์

หากอาหารมี:

`category === "limited"`

ให้ถือว่าเป็นอาหารที่ควรรับประทานแต่น้อย

ห้ามตรวจคำตอบจากชื่อไฟล์หรือข้อความที่แสดงบนหน้าจอ ให้ตรวจจากข้อมูลใน Object

25. คะแนนอาหารที่มีประโยชน์

เมื่อวางอาหารที่มีประโยชน์ลงตะกร้า:

- ได้รับ 100 คะแนน
- แสดง Floating Score **+100**
- แสดงข้อความ **ได้รับ 100 คะแนน**
- แสดงชื่ออาหาร
- แสดงชื่อหมู่อาหาร
- เล่นเสียง Bell หรือ Chime
- แสดง Particle สีเขียว
- เพิ่ม **correctCount**
- นำอาหารชิ้นนั้นออกจากฉาก

- ป้องกันการนับ Entity เดิมซ้ำ
- สุ่มอาหารใหม่เข้ามาแทน

ตัวอย่างข้อความ:

+100 คะแนน

ข้าว – หมู 2 คาร์โบไฮเดรต

หากเป็นหมู่ที่ผู้เล่นยังไม่เคยเก็บ ให้เพิ่มหมูนั้นลงใน:

level5State.collectedGroups

และแสดงข้อความ:

เก็บหมู่ที่ 2 สำเร็จ!

หากเป็นอาหารหมู่เดิมที่เก็บแล้ว:

- ยังคงได้ 100 คะแนน
- แต่ห้ามเพิ่มจำนวนหมูซ้ำ
- แสดงข้อความ **หมูนี้เก็บแล้ว**

26. คะแนนอาหารที่ควรรับประทานครึ่งน้อย

เมื่อวางอาหารที่ควรรับประทานครึ่งน้อยลงตะกร้า:

- หัก 30 คะแนน
- คะแนนด้านต้องไม่ต่ำกว่า 0
- แสดง Floating Score **-30**
- แสดงข้อความ **หัก 30 คะแนน**
- แสดงข้อความ **ควรรับประทานครึ่งน้อย**
- เล่นเสียง Buzzer
- แสดง Particle สีเทา
- เพิ่ม **errorCount**
- นำอาหารชิ้นนั้นออกจากฉาก
- สุ่มอาหารใหม่เข้ามาแทน
- ห้ามเพิ่ม Progress อาหาร 5 หมู

ตัวอย่าง:

```
gameState.levelScore = Math.max(
    0,
    gameState.levelScore - 30
);
```

27. ป้องกันการนับอาหารซ้ำ

อาหารทุกชิ้นต้องมี Entity ID ที่ไม่ซ้ำกัน เช่น:

instanceId: "food-1730000000-1"

ก่อนให้คะแนน ให้ตรวจว่า Entity ID ยังไม่อยู่ใน:

level5State.collectedFoodIds

หลังตรวจคำตอบแล้ว ให้เพิ่ม ID ลงใน Set และลบ Entity ออกจากฉากทันที
ห้ามให้ Pointer Up, Pinch Release หรือ Game Loop เรียกตรวจอาหารขึ้นเดียวกันซ้ำ

28. เงื่อนไขผ่าน

ผู้เล่นผ่านด่านเมื่อ:

`level5State.collectedGroups.size === 5`

และเวลาเหลือมากกว่า 0

เมื่อผ่าน:

- หยุดรับ Input ของด่าน
- หยุด Timer
- หยุด Spawn
- ยกเลิก Drag State
- เพิ่มสถานะ `levelFinished`
- เรียก `finishLevel("passed")` เพียงครั้งเดียว
- เล่นเสียง Victory หรือ Chime
- แสดง Particle สีเขียว
- แสดงข้อความ **อาหารครบ 5 หมูแล้ว!**

ห้ามใช้จำนวนอาหารที่เก็บทั้งหมดเป็นเงื่อนไขผ่าน

ตัวอย่างที่ไม่ถูกต้อง:

`itemsCollected >= 5`

ตัวอย่างที่ถูกต้อง:

`collectedGroups.size >= 5`

29. เงื่อนไขไม่ผ่าน

ผู้เล่นไม่ผ่านเมื่อ:

เวลาเป็น 0

และ

เก็บอาหารได้ไม่ครบทั้ง 5 หมู

ก่อนจบด่าน ให้ตรวจสอบอีกครั้งว่า:

`level5State.collectedGroups.size < 5`

จากนั้นเรียก:

`finishLevel("failed")`

เพียงครั้งเดียว

30. เกณฑ์ดาว

กำหนดเกณฑ์ดาวดังนี้:

3 ดาว

- เก็บอาหารครบทั้ง 5 หมู

- ไม่เลือกอาหารที่ควรรับประทานแต่น้อย
- `errorCount === 0`
- มีเวลาเหลืออย่างน้อย 15 วินาที

2 ดาว

- เก็บอาหารครบทั้ง 5 หมู่
- เลือกผิดไม่เกิน 1 ครั้ง
- `errorCount <= 1`

1 ดาว

- เก็บอาหารครบทั้ง 5 หมู่
- เลือกผิดตั้งแต่ 2 ครั้งขึ้นไป

ลำดับตรวจดาวต้องตรวจ 3 ดาวก่อน 2 ดาว และ 1 ดาวตามลำดับ

31. Result Screen ของด่านที่ 5

หลังจบด่าน ให้แสดงข้อมูล:

- ผลการเล่น
- คะแนนด่าน
- เวลาเหลือ
- จำนวนอาหารที่เลือกถูก
- จำนวนครั้งที่เลือกผิด
- จำนวนหมู่อาหารที่เก็บสำเร็จ
- รายชื่อหมู่ที่เก็บสำเร็จ
- รายชื่อหมู่ที่ยังขาด
- จำนวนดาว
- ข้อความความรู้

หากผ่าน ให้แสดง:

ยอดเยี่ยม! คุณเลือกอาหารได้ครบทั้ง 5 หมู่ ควรรับประทานอาหารที่หลากหลายและรับประทานอาหารบางชนิดในปริมาณที่เหมาะสม

หากไม่ผ่าน ให้แสดง:

ยังเก็บอาหารไม่ครบทั้ง 5 หมู่ ลองสังเกตว่าอาหารแต่ละชนิดอยู่ในหมู่ใด แล้วลองอีกครั้ง

ข้อความหลักหลังผ่าน:

ยอดเยี่ยม! ควรรับประทานอาหารที่มีประโยชน์และหลีกเลี่ยงอาหารที่ควรรับประทานแต่น้อย

32. ข้อกำหนดด้านเนื้อหาสุขภาพ

ห้ามใช้ข้อความที่กล่าวอ้างว่า:

- อาหารชนิดใดสามารถป้องกัน PM 2.5 ได้โดยตรง
- อาหารชนิดใดสามารถกำจัดฝุ่นในร่างกายได้
- อาหารชนิดใดสามารถรักษาผลกระทบจาก PM 2.5 ได้
- การรับประทานอาหารสามารถทดแทนหน้ากากได้
- การรับประทานอาหารสามารถทดแทนการลดการสัมผัสฝุ่นได้
- การรับประทานอาหารสามารถทดแทนคำแนะนำจากแพทย์ได้
- ผู้ที่ผ่านด่านมีสุขภาพแข็งแรงจริง

ให้เน้นว่า:

- ควรรับประทานอาหารอย่างหลากหลาย
- ควรรับประทานให้เหมาะสม

- อาหารแต่ละหมู่ให้สารอาหารแตกต่างกัน
 - อาหารบางชนิดควรรับประทานแต่น้อย
 - เกมนี้เป็นกิจกรรมเพื่อการเรียนรู้เท่านั้น
-

33. การวาดมือ AR บน Canvas

เมื่อเล่นด่านที่ 5 ในโหมด AR ให้แสดง:

- จุดปลายนิ้วโป้ง
- จุดปลายนิ้วชี้
- จุดปลายนิ้วกลาง
- จุดปลายนิ้วนาง
- จุดปลายนิ้วก้อย
- Pinch Cursor
- เส้นเชื่อมระหว่างนิ้วโป้งกับนิ้วชี้
- สถานะ พร้อมจับ
- สถานะ กำลังจับ
- สถานะ ปล่อย

เมื่อจับอาหาร:

- เปลี่ยน Pinch Cursor เป็นสีเขียวหรือ Glow
- ขยายวง Cursor เล็กน้อย
- แสดงชื่ออาหารที่กำลังจับ
- ห้ามใช้เอฟเฟกต์กะพริบรุนแรง

เมื่อเปิด Reduced Motion ให้ลด Animation และ Glow

34. Input State กลาง

ให้ทุกโหมดส่งข้อมูลเข้าสู่ Input State กลางชุดเดิม

ตัวอย่าง:

inputState.x

inputState.y

inputState.active

inputState.pressed

inputState.justPressed

inputState.released

inputState.gesture

สำหรับ AR ด่านที่ 5:

Pinch เริ่มต้น → justPressed = true

Pinch ค้าง → pressed = true

Pinch ปล่อย → released = true

ค่า **justPressed** และ **released** ต้องมีผลเพียงหนึ่ง Frame แล้วรีเซ็ตหลัง **updateGame()**

ห้ามสร้าง Logic ด่านที่ 5 แยกเป็น 3 ชุดสำหรับ AR, Mouse และ Touch

35. ฟังก์ชันที่ต้องมี Logic ใช้งานจริง

ให้เพิ่มหรือปรับฟังก์ชันอย่างน้อยดังนี้:

initLevel5()
resetLevel5State()
createLevel5FoodData()
buildLevel5SpawnQueue()
spawnLevel5Food()
getLevel5SpawnPosition()
updateLevel5(deltaTime)
renderLevel5()
handleLevel5GrabStart()
updateLevel5Dragging()
handleLevel5Release()
handleLevel5FoodDrop(food)
addCollectedFoodGroup(groupId)
updateLevel5ProgressUI()
showLevel5Feedback()
calculateLevel5Stars()
cleanupLevel5()
detectLevel5Pinch()
updateFiveFingerLandmarks()
drawFiveFingerTracking()

ทุกฟังก์ชันต้องมี Logic ใช้งานจริง
ห้ามมี:

TODO

Pseudocode

ฟังก์ชันว่าง

ส่วนอื่นเหมือนเดิม

ให้นำโค้ดไปเติมเอง

36. Resource Cleanup ของด่านที่ 5

เมื่อออกจากด่าน Restart กลับแผนที่ หรือเปลี่ยนหน้า ให้:

- ยกเลิก Drag State
- ปลอ่ย Pointer Capture
- รีเซ็ต Pinch State
- รีเซ็ต **draggingFood**
- ล้าง Food Entities
- ล้าง Particle ของด่าน
- ล้าง Feedback Timer
- ล้าง **collectedGroups**
- ล้าง **collectedFoodIds**
- หยุด Speech ของด่าน
- ไม่เปิดหรือปิดกล้องซ้ำโดยไม่จำเป็น

- ไม่สร้าง MediaPlayer Loop ใหม่
- ไม่สร้าง Event Listener ซ้ำ
- ไม่สร้าง Game Loop ซ้อน

หากยังอยู่ในโหมด AR และไปจาก Mission Map เข้าด่านที่ 5 ให้ใช้ Stream ที่ยังทำงานอยู่ ห้ามเรียก `getUserMedia()` ซ้ำ

37. Acceptance Criteria ด้านข้อมูลอาหาร

ก่อนส่งโค้ด ให้ตรวจสอบว่า:

- มีอาหารหมู่ที่ 1 ครบตามรายการ
 - มีอาหารหมู่ที่ 2 ครบตามรายการ
 - มีอาหารหมู่ที่ 3 ครบตามรายการ
 - มีอาหารหมู่ที่ 4 ครบตามรายการ
 - มีอาหารหมู่ที่ 5 ครบตามรายการ
 - มีอาหารที่ควรรับประทานแต่น้อยครบตามรายการ
 - Asset Path ถูกต้อง
 - ตัวพิมพ์เล็กและใหญ่ตรงกับชื่อไฟล์
 - ภาพที่โหลดไม่ได้มี Fallback
 - ไม่มี Broken Image Icon
-

38. Acceptance Criteria ด้าน Gameplay

ตรวจสอบว่า:

- เวลาเริ่มต้นเป็น 45 วินาที
 - เริ่มด่านแล้วมีอาหารครบทุกหมู่ให้ผู้เล่นสามารถผ่านได้
 - อาหารไม่วางทับกันจนจับไม่ได้
 - วางอาหารที่มีประโยชน์ได้ +100
 - วางอาหารที่ควรรับประทานแต่น้อยได้ -30
 - คะแนนไม่ต่ำกว่า 0
 - หมู่เดิมไม่เพิ่ม Progress ซ้ำ
 - อาหารชิ้นเดิมไม่ถูกนับซ้ำ
 - เก็บครบ 5 หมู่แล้วผ่านทันที
 - เก็บอาหาร 5 ชิ้นจากหมู่เดียวกันแล้วต้องยังไม่ผ่าน
 - เวลาเป็น 0 และไม่ครบ 5 หมู่ต้องไม่ผ่าน
 - `finishLevel()` ทำงานเพียงครั้งเดียว
 - Restart แล้ว Progress กลับเป็น 0/5
 - Pause แล้วเวลาและการลากหยุดจริง
-

39. Acceptance Criteria ด้าน AR

ตรวจสอบว่า:

- แสดงปลายนิ้วทั้ง 5 นิ้ว
- ใช้ Landmark 4, 8, 12, 16 และ 20 จริง
- Pinch Cursor อยู่ระหว่างนิ้วโป้งและนิ้วชี้
- Pinch เริ่มบน Hitbox อาหารแล้วจับได้
- Pinch นอกอาหารแล้วไม่จับวัตถุ
- ลากอาหารตามมืออย่างลื่นไหล

- ปล่อนิ้วแล้ววางอาหารได้
- Release Threshold มากกว่า Pinch Threshold
- Gesture ไม่สลับจับ–ปล่อยอย่างรวดเร็ว
- ไม่ใช่ Fist เป็นคำสั่งจับ
- ไม่ใช่ Open Hand เป็นคำสั่งปล่อย
- ไม่จับอาหารหลายชิ้นพร้อมกัน
- เมื่อมือหายชั่วคราว เกมไม่ค้าง
- เมื่อมือหายนาน อาหารกลับตำแหน่งเดิม
- ไม่มี `NotReadableError`
- ไม่มีการเปิดกล้องซ้ำ
- ไม่มี MediaPipe Processing Loop ซ้ำ

40. Acceptance Criteria ด้าน Mouse และ Touch

ตรวจสอบว่า:

- Mouse กดค้างแล้วลากได้
- Mouse ปล่อยในตะกร้าแล้วตรวจคำตอบ
- Mouse ปล่อยนอกตะกร้าแล้วอาหารกลับที่เดิม
- Touch แตะค้างแล้วลากได้
- Touch ปล่อยในตะกร้าแล้วตรวจคำตอบ
- Touch ปล่อยนอกตะกร้าแล้วอาหารกลับที่เดิม
- การควบคุม Touch ไม่เลื่อนหน้า Gameplay
- Mouse และ Touch เล่นด่านที่ 5 ผ่านได้โดยไม่ต้องใช้กล้อง
- การปรับ AR ไม่ทำให้ Mouse และ Touch เสีย

41. รูปแบบการส่งมอบ

ให้ตอบกลับด้วยโค้ด `index.html` ฉบับสมบูรณ์ โดยมีเงื่อนไขดังนี้:

1. ใช้ไฟล์ล่าสุดเป็นฐาน
2. ห้ามสร้างเกมใหม่ทั้งหมด
3. ห้ามลบ Gameplay ของด่านที่ 1–4
4. ห้ามลบระบบ AR Lifecycle ที่แก้จากคำสั่งที่ 8
5. แก้เฉพาะส่วนที่เกี่ยวข้องกับด่านที่ 5 และ Landmark Input
6. ต้องรองรับ AR, Mouse และ Touch
7. ต้องใช้ Game State และ Input State กลาง
8. ต้องมีอาหารครบทั้ง 5 หมู่ตามรายการ
9. ต้องมีอาหารที่ควรรับประทานแต่น้อยตามรายการ
10. ต้องเก็บครบ 5 หมู่จึงผ่าน
11. ต้องใช้ Pinch และ Release เป็นคำสั่งจับและปล่อย
12. ต้องติดตามและแสดงตำแหน่งนิ้วทั้ง 5 นิ้ว
13. ห้ามมี TODO
14. ห้ามมี Pseudocode
15. ห้ามมีฟังก์ชันว่าง
16. ห้ามตัดส่วนโค้ด
17. ต้องไม่มี SyntaxError

18. ต้องไม่มี ReferenceError
19. ต้องไม่มี ID ที่ JavaScript เรียกแต่ไม่มีใน HTML
20. ต้องไม่มี Event Listener ซ้ำ
21. ต้องไม่มี Timer ซ้ำ
22. ต้องไม่มี Game Loop ซ้อน
23. ต้องไม่มี Camera Stream ซ้ำ
24. Asset เสียต้องมี Fallback
25. ต้องพร้อมใช้งานผ่าน GitHub Pages
26. ตอบด้วย Code Block ภาษา HTML เพียง Code Block เดียว
27. ห้ามอธิบายนอก Code Block